Airbnb Analysis

Anish

2026-10-13

Aim of the project

To investigate what drives the nightly price of short-term Airbnb listings in Sydney, by modelling the distribution of prices, testing whether prices differ between key groups of listings, and building a regression model to predict price from property, host and location characteristics.

Setup

This section loads the R packages used throughout the analysis. **dplyr** is used to select, filter and summarise data, **ggplot2** to create plots, and **scales** to format axis labels (e.g. $1,000). We also define a small function to calculate skewness, since base R does not include one. Skewness measures how asymmetric a distribution is, and tells us whether a transformation (such as a log) is needed.

```
library(dplyr)
library(ggplot2)
library(scales)

# Sample skewness (0 = symmetric, > 0 = long right tail, < 0 = long left tail)
skewness <- function(x) {
  x <- x[!is.na(x)]
  mean((x - mean(x))^3) / sd(x)^3
}

# Colours (colour-blind-safe blue and orange)
col_main  <- "#2a78d6"
room_cols <- c("Entire home/apt" = "#2a78d6", "Private room" = "#eb6834")

# One plot style for every chart
theme_set(
  theme_minimal(base_size = 12) +
    theme(panel.grid.minor = element_blank(),
          plot.title = element_text(face = "bold"),
          plot.title.position = "plot")
)

# Whole-dollar axis labels, e.g. $1,000 rather than $1,000.00
dollar0 <- label_dollar(accuracy = 1)
```

1. Read the data

The data come from **Inside Airbnb** (insideairbnb.com), an independent project that publishes information collected from public Airbnb listings. We use the Sydney "Detailed Listings" file, collected on 17/09/2026. Each row is one Airbnb listing, and the raw file contains 20,833 listings and 90 variables.

Before cleaning, we check the size of the data, the type of the key variables and how many values are missing. This tells us what needs to be fixed before analysis.

```
raw <- read.csv("listings.csv")
dim(raw)
```

```
## [1] 20833    90
```

```
head(raw[, c("room_type", "price", "accommodates", "bedrooms", "bathrooms_text")])
```

```
##         room_type   price accommodates bedrooms bathrooms_text
## 1 Entire home/apt $140.00            2        1         1 bath
## 2 Entire home/apt $407.62            2        1      1.5 baths
## 3    Private room  $63.99            1        1  1 shared bath
## 4    Private room $221.00            2        1 1 private bath
## 5 Entire home/apt                    2        1         1 bath
## 6 Entire home/apt $227.50            4        1         1 bath
```

```
str(raw$price)
```

```
##  chr [1:20833] "$140.00" "$407.62" "$63.99" "$221.00" "" "$227.50" ...
```

```
sum(raw$price == "")
```

```
## [1] 3112
```

```
table(raw$room_type)
```

```
##
## Entire home/apt      Hotel room    Private room     Shared room
##           16445              58            4282              48
```

2. Select variables and fix their types

Most of the 90 columns are text, web links or ID numbers that are not useful for statistical analysis, so we keep only the 17 variables related to price, property size, location, host and reviews.

Some variables also need converting before R can use them:

- **price** is stored as text (e.g. "1,250.00"), *so* the "" and "," are removed and it is converted to a number. R cannot calculate means or draw histograms from text.
- **bathrooms** is empty in this snapshot, so the number of bathrooms is taken from the text column `bathrooms_text` (e.g. "1.5 baths" becomes 1.5). We also record whether the bathroom is shared.
- **superhost** is stored as "t"/"f", so it is recoded to "Superhost" / "Not superhost" to make tables and plots easier to read.

```r
# Bathrooms: "1.5 baths" -> "1.5", "Half-bath" -> "0.5", "No bathroom" -> "0"
bath_number <- sub(" .*", "", raw$bathrooms_text)
bath_number[grepl("half", raw$bathrooms_text, ignore.case = TRUE)] <- "0.5"
bath_number[raw$bathrooms_text %in% "No bathroom"] <- "0"

airbnb <- data.frame(
  id                = raw$id,
  price             = as.numeric(gsub("[$,]", "", raw$price)),
  room_type         = raw$room_type,
  property_type     = raw$property_type,
  lga               = raw$neighbourhood_cleansed,
  accommodates      = raw$accommodates,
  bedrooms          = raw$bedrooms,
  bathrooms         = as.numeric(bath_number),
  shared_bath       = ifelse(grepl("shared", raw$bathrooms_text, ignore.case = TRUE),
                             "Shared", "Private"),
  superhost         = ifelse(raw$host_is_superhost == "t", "Superhost", "Not superhost"),
  minimum_nights    = raw$minimum_nights,
  availability_365  = raw$availability_365,
  number_of_reviews = raw$number_of_reviews,
  reviews_last_12m  = raw$number_of_reviews_ltm,
  review_rating     = raw$review_scores_rating,
  latitude          = raw$latitude,
  longitude         = raw$longitude
)

str(airbnb)
```

```
## 'data.frame':    20833 obs. of  17 variables:
##  $ id                : num  1877472 1886610 11156 15253 44545 ...
##  $ price             : num  140 408 64 221 NA ...
##  $ room_type         : chr  "Entire home/apt" "Entire home/apt" "Private room" "Private room" ...
##  $ property_type     : chr  "Entire guesthouse" "Entire rental unit" "Private room in rental unit" "P
##  $ lga               : chr  "Pittwater" "Waverley" "Sydney" "Sydney" ...
##  $ accommodates      : int  2 2 1 2 2 4 4 2 5 2 ...
##  $ bedrooms          : int  1 1 1 1 1 1 2 1 2 1 ...
##  $ bathrooms         : num  1 1.5 1 1 1 1 2.5 1 1 1 ...
##  $ shared_bath       : chr  "Private" "Private" "Shared" "Private" ...
##  $ superhost         : chr  "Superhost" "Superhost" "Not superhost" "Superhost" ...
##  $ minimum_nights    : int  1 1 90 1 3 2 1 2 6 1 ...
##  $ availability_365  : int  218 211 364 188 4 175 297 281 210 17 ...
##  $ number_of_reviews: int  478 575 193 689 92 484 135 61 22 373 ...
##  $ reviews_last_12m  : int  39 68 0 57 7 36 15 12 0 18 ...
##  $ review_rating     : num  4.92 4.91 4.59 4.57 4.9 4.65 4.96 4.87 4.45 4.95 ...
##  $ latitude          : num  -33.7 -33.9 -33.9 -33.9 -33.9 ...
##  $ longitude         : num  151 151 151 151 151 ...
```

```r
# Keep a copy of every priced listing (used later for a before/after plot)
priced_all <- filter(airbnb, !is.na(price))
```

3. Cleaning

The aim of cleaning is to define a clear population for the analysis: **active, short-term Airbnb listings in Sydney, offered as an entire home or a private room.** Each step removes listings that fall outside this population, and the number of rows remaining is recorded after every step so the process is transparent.

1. **Has a price:** listings without a price cannot be analysed.
2. **Entire home or private room:** hotel rooms and shared rooms together make up fewer than 100 listings, too few for reliable comparisons.
3. **Active listings:** listings with no reviews in the past 12 months are unlikely to be genuinely booked, and some carry unrealistic placeholder prices (up to $96,888 per night).
4. **Short-term listings:** listings with a minimum stay over 30 nights are monthly rentals, which form a different market from short-term stays.

```
steps <- data.frame(step = "Raw data", rows = nrow(airbnb))

# Step 1: keep listings that have a price
airbnb <- filter(airbnb, !is.na(price))
steps <- rbind(steps, data.frame(step = "1. Has a price", rows = nrow(airbnb)))

# Step 2: keep the two main room types (hotel + shared rooms are under 100 listings)
airbnb <- filter(airbnb, room_type %in% c("Entire home/apt", "Private room"))
steps <- rbind(steps, data.frame(step = "2. Entire home or private room", rows = nrow(airbnb)))

# Step 3: keep active listings (at least one review in the last 12 months)
airbnb <- filter(airbnb, reviews_last_12m >= 1)
steps <- rbind(steps, data.frame(step = "3. Active (1+ review in last 12 months)", rows = nrow(airbnb)))

# Step 4: keep short-term listings (minimum stay of 30 nights or less)
airbnb <- filter(airbnb, minimum_nights <= 30)
steps <- rbind(steps, data.frame(step = "4. Short-term (minimum stay <= 30 nights)", rows = nrow(airbnb)))

steps
```

```
##                                       step  rows
## 1                                 Raw data 20833
## 2                            1. Has a price 17721
## 3            2. Entire home or private room 17625
## 4   3. Active (1+ review in last 12 months) 13449
## 5 4. Short-term (minimum stay <= 30 nights) 13259
```

Removing extreme prices

Price is strongly right-skewed. The usual boxplot rule (1.5 × IQR) applied to raw price would flag every listing above about $900, removing hundreds of genuine luxury homes. Instead, the rule is applied to **log price**, where the distribution is roughly symmetric, and a wider **3 × IQR** fence is used so that only truly extreme values are removed. The fence is then converted back to dollars. The listings flagged as outliers are inspected before they are removed.

```
log_q1  <- as.numeric(quantile(log(airbnb$price), 0.25))
log_q3  <- as.numeric(quantile(log(airbnb$price), 0.75))
log_iqr <- log_q3 - log_q1
```

```r
lower_fence <- exp(log_q1 - 3 * log_iqr)
upper_fence <- exp(log_q3 + 3 * log_iqr)
round(c(lower = lower_fence, upper = upper_fence), 2)
```

```
##   lower   upper
##   21.13 5270.76
```

```r
# Look at the listings that will be removed
outliers <- filter(airbnb, price < lower_fence | price > upper_fence)
outliers %>%
  arrange(desc(price)) %>%
  select(price, room_type, property_type, lga, accommodates, bedrooms, reviews_last_12m)
```

```
##       price       room_type      property_type        lga accommodates bedrooms
## 1  12224.00 Entire home/apt Entire rental unit     Sydney            6        3
## 2  12172.33 Entire home/apt Entire rental unit     Auburn            5        2
## 3  12137.00 Entire home/apt Entire rental unit     Sydney            3        1
## 4  11717.67 Entire home/apt Entire rental unit     Sydney            2        1
## 5  10696.00 Entire home/apt Entire rental unit     Sydney            4        2
## 6   9151.00 Entire home/apt        Entire home     Sydney           14        7
## 7   8749.75 Entire home/apt Entire rental unit     Sydney            2        0
## 8   6700.00 Entire home/apt       Entire villa   Randwick           10        5
## 9   6693.60 Entire home/apt        Entire home   Waverley           10        5
## 10  6521.74 Entire home/apt        Entire home   Randwick            8        4
## 11  6000.00 Entire home/apt Entire rental unit Leichhardt            2        1
## 12  5509.20 Entire home/apt Entire rental unit   Waverley            6        3
## 13  5306.00 Entire home/apt        Entire home     Sydney           10        4
##    reviews_last_12m
## 1                11
## 2                28
## 3                 3
## 4                15
## 5                 6
## 6                 4
## 7                46
## 8                 1
## 9                 1
## 10                3
## 11                3
## 12                1
## 13               14
```

```r
# Remove them
airbnb <- filter(airbnb, price >= lower_fence, price <= upper_fence)
steps <- rbind(steps, data.frame(step = "5. Price inside log-scale 3 x IQR fence", rows = nrow(airbnb)))
```

Missing values

A small number of listings (27) are missing the number of bedrooms or bathrooms. These variables are needed for the regression analysis, so these rows are removed. Because so few rows are affected, this has a negligible impact on the results.

```r
# Step 6: drop the few rows missing bedrooms or bathrooms
airbnb <- filter(airbnb, !is.na(bedrooms), !is.na(bathrooms))
steps <- rbind(steps, data.frame(step = "6. Bedrooms and bathrooms recorded", rows = nrow(airbnb)))

# How many rows each step removed
steps$removed <- c(0, -diff(steps$rows))
steps
```

```
##                                       step  rows removed
## 1                                 Raw data 20833       0
## 2                             1. Has a price 17721    3112
## 3            2. Entire home or private room 17625      96
## 4   3. Active (1+ review in last 12 months) 13449    4176
## 5 4. Short-term (minimum stay <= 30 nights) 13259     190
## 6   5. Price inside log-scale 3 x IQR fence 13246      13
## 7        6. Bedrooms and bathrooms recorded 13219      27
```

4. Derived variables and save

Two new variables are created:

- **log_price**, the natural log of price. Price is heavily right-skewed, and taking logs makes its distribution close to symmetric. Log price is used in plots, in distribution fitting (log-normal model) and as the response in the regression.
- **bedroom_group**, which groups listings into Studio, 1, 2, 3, 4 and 5+ bedrooms. Listings with 5 or more bedrooms are combined because there are very few of each. It is stored as an ordered factor so groups appear in a sensible order in tables and plots.

The cleaned data are saved as a CSV file so all group members can work from the same dataset.

```r
# Log of price (used for plots, tests and regression later)
airbnb$log_price <- log(airbnb$price)

# Group bedrooms: 0 = Studio, 5 or more = "5+"
airbnb$bedroom_group <- ifelse(airbnb$bedrooms == 0, "Studio",
                        ifelse(airbnb$bedrooms >= 5, "5+", as.character(airbnb$bedrooms)))
airbnb$bedroom_group <- factor(airbnb$bedroom_group,
                               levels = c("Studio", "1", "2", "3", "4", "5+"))
table(airbnb$bedroom_group)
```

```
##
## Studio      1      2      3      4     5+
##    687   6033   3714   1576    759    450
```

```r
# Save the clean data so the group can use it
write.csv(airbnb, "airbnb_sydney_clean.csv", row.names = FALSE)
dim(airbnb)
```

```
## [1] 13219    19
```

5. Descriptive statistics

Descriptive statistics summarise the centre (mean, median), spread (standard deviation, quartiles) and shape (skewness) of each variable. They show which variables are skewed and which groups differ, which guides the choice of distributions, hypothesis tests and regression models later in the analysis.

Numeric variables

Summary statistics for each numeric variable. Comparing the mean with the median, and checking the skewness, shows which variables are skewed.

```
num_vars <- c("price", "log_price", "accommodates", "bedrooms", "bathrooms",
              "minimum_nights", "availability_365", "number_of_reviews", "review_rating")

num_summary <- data.frame(
  n        = sapply(airbnb[num_vars], length),
  mean     = sapply(airbnb[num_vars], mean),
  sd       = sapply(airbnb[num_vars], sd),
  min      = sapply(airbnb[num_vars], min),
  q1       = sapply(airbnb[num_vars], quantile, 0.25),
  median   = sapply(airbnb[num_vars], median),
  q3       = sapply(airbnb[num_vars], quantile, 0.75),
  max      = sapply(airbnb[num_vars], max),
  skewness = sapply(airbnb[num_vars], skewness)
)
knitr::kable(num_summary, digits = 2)
```

	n	mean	sd	min	q1	median	q3	max	skewness
price	13219	429.55	393.35	41.50	225.00	341.00	494.95	5076.80	4.43
log_price	13219	5.81	0.69	3.73	5.42	5.83	6.20	8.53	0.09
accommodates	13219	4.09	2.52	1.00	2.00	4.00	6.00	16.00	1.48
bedrooms	13219	1.79	1.18	0.00	1.00	1.00	2.00	11.00	1.44
bathrooms	13219	1.45	0.74	0.00	1.00	1.00	2.00	12.00	2.38
minimum_nights	13219	2.07	2.34	1.00	1.00	2.00	2.00	30.00	6.58
availability_365	13219	187.14	110.38	1.00	86.00	177.00	290.00	365.00	0.02
number_of_reviews	13219	54.61	85.16	1.00	7.00	24.00	66.00	1166.00	3.81
review_rating	13219	4.73	0.41	1.00	4.66	4.84	4.96	5.00	-4.51

Categorical variables

Counts and percentages for each category, to show how listings are split between groups.

```
cat_counts <- rbind(
  data.frame(variable = "room_type",   table(level = airbnb$room_type)),
  data.frame(variable = "superhost",   table(level = airbnb$superhost)),
  data.frame(variable = "shared_bath", table(level = airbnb$shared_bath))
)
cat_counts$percent <- round(100 * cat_counts$Freq / nrow(airbnb), 1)
knitr::kable(cat_counts)
```

variable	level	Freq	percent
room_type	Entire home/apt	10774	81.5
room_type	Private room	2445	18.5
superhost	Not superhost	7507	56.8
superhost	Superhost	5712	43.2
shared_bath	Private	11734	88.8
shared_bath	Shared	1485	11.2

Price by group

Price is compared across room types, bedroom groups, Superhost status and bathroom type. Because price is skewed, the **median and quartiles** are the main measures used: the mean is pulled upwards by a small number of very expensive listings.

```
# By room type
airbnb %>%
  group_by(room_type) %>%
  summarise(n = n(), mean = mean(price), sd = sd(price),
            q1 = quantile(price, 0.25), median = median(price), q3 = quantile(price, 0.75)) %>%
  knitr::kable(digits = 2)
```

room_type	n	mean	sd	q1	median	q3
Entire home/apt	10774	492.87	407.85	285.50	384.95	543.5
Private room	2445	150.51	89.48	98.68	129.00	175.5

```
# By number of bedrooms
airbnb %>%
  group_by(bedroom_group) %>%
  summarise(n = n(), mean = mean(price), sd = sd(price),
            q1 = quantile(price, 0.25), median = median(price), q3 = quantile(price, 0.75)) %>%
  knitr::kable(digits = 2)
```

bedroom_group	n	mean	sd	q1	median	q3
Studio	687	271.45	111.53	201.00	255.40	317.45
1	6033	254.55	177.94	144.00	237.00	326.50
2	3714	455.40	268.13	322.00	408.50	516.63
3	1576	659.19	414.63	421.41	552.88	768.70
4	759	956.21	642.57	533.24	743.50	1174.25
5+	450	1111.19	800.58	567.55	838.36	1421.75

```
# By Superhost status
airbnb %>%
  group_by(superhost) %>%
  summarise(n = n(), mean = mean(price), sd = sd(price),
            q1 = quantile(price, 0.25), median = median(price), q3 = quantile(price, 0.75)) %>%
  knitr::kable(digits = 2)
```

superhost	n	mean	sd	q1	median	q3
Not superhost	7507	420.86	387.06	209.92	333.52	488.00
Superhost	5712	440.97	401.21	238.50	349.50	502.21

```
# By shared vs private bathroom
airbnb %>%
  group_by(shared_bath) %>%
  summarise(n = n(), mean = mean(price), sd = sd(price),
            q1 = quantile(price, 0.25), median = median(price), q3 = quantile(price, 0.75)) %>%
  knitr::kable(digits = 2)
```

shared_bath	n	mean	sd	q1	median	q3
Private	11734	467.77	400.89	263.5	367.00	521.46
Shared	1485	127.56	68.17	88.2	112.06	150.00

Price by local government area

Median price for each local government area (LGA) with at least 100 listings. Areas with fewer listings are excluded because their medians would be unreliable.

```
price_lga <- airbnb %>%
  group_by(lga) %>%
  summarise(n = n(), median_price = median(price)) %>%
  filter(n >= 100) %>%
  arrange(desc(median_price))

knitr::kable(price_lga, digits = 2)
```

lga	n	median_price
Pittwater	483	585.00
Manly	570	451.50
Woollahra	339	451.00
Waverley	1071	424.55
Mosman	145	408.00
North Sydney	542	383.12
Warringah	463	362.00
Sydney	3445	360.00
Sutherland Shire	359	353.00
Randwick	632	352.64
Auburn	542	336.44
Botany Bay	220	336.25
Camden	110	335.66
Canada Bay	160	330.28
Willoughby	180	314.00
Ryde	285	307.33
Leichhardt	275	304.72
Campbelltown	107	289.00
Strathfield	101	281.05

lga	n	median_price
Hornsby	268	277.35
Penrith	167	260.90
Liverpool	157	259.50
Blacktown	228	258.50
Ku-Ring-Gai	151	254.00
Burwood	137	250.00
Parramatta	306	244.83
Marrickville	334	244.68
The Hills Shire	241	242.00
Rockdale	264	223.33
Fairfield	220	215.58
Canterbury	114	195.35
Bankstown	221	157.99

Correlations

Spearman correlations measure how strongly pairs of variables move together. Spearman is used instead of Pearson because it is based on ranks, so it is not distorted by skewed variables or extreme values.

```
cor_vars <- c("price", "accommodates", "bedrooms", "bathrooms", "minimum_nights",
              "availability_365", "number_of_reviews", "review_rating")

cor_mat <- cor(airbnb[cor_vars], method = "spearman")
knitr::kable(cor_mat, digits = 2)
```

	price	accommodates	bedrooms	bathrooms	minimum_nights	availability_365	number_of_reviews	review_rating
price	1.00	0.70	0.67	0.43	0.11	0.00	-0.08	0.24
accommodates	0.70	1.00	0.86	0.57	0.05	0.01	-0.07	0.10
bedrooms	0.67	0.86	1.00	0.65	0.13	0.00	-0.14	0.12
bathrooms	0.43	0.57	0.65	1.00	0.07	0.00	-0.18	0.05
minimum_nights	0.11	0.05	0.13	0.07	1.00	-0.07	-0.11	0.11
availability_365	0.00	0.01	0.00	0.00	-0.07	1.00	-0.03	-0.05
number_of_reviews	-0.08	-0.07	-0.14	-0.18	-0.11	-0.03	1.00	-0.12
review_rating	0.24	0.10	0.12	0.05	0.11	-0.05	-0.12	1.00

6. Visualisations

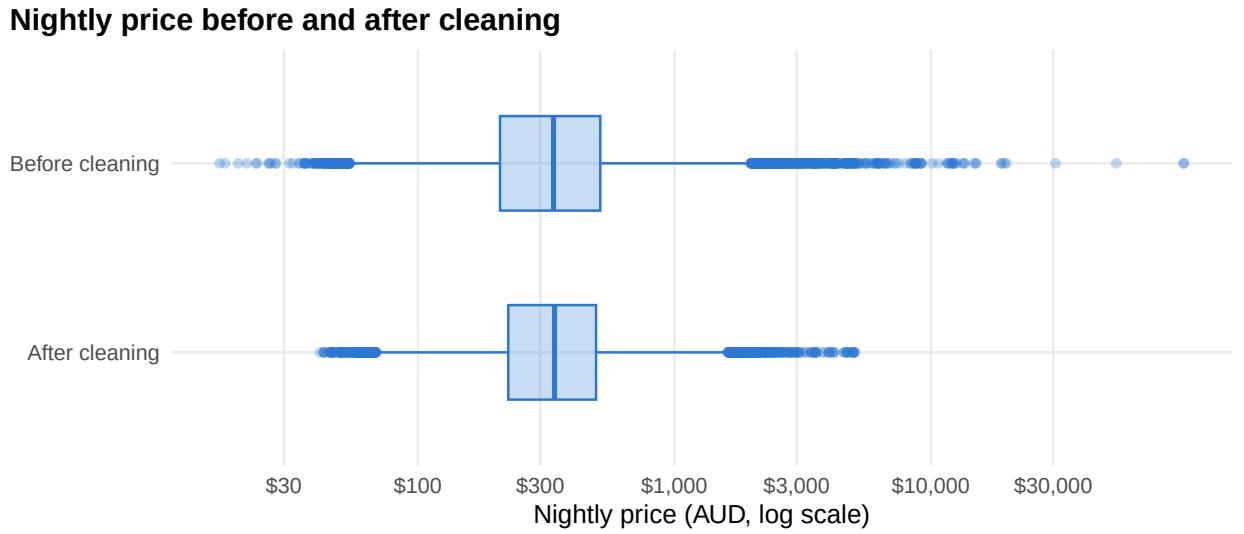
Plots show the shape of each distribution and the differences between groups more clearly than tables. Because price is right-skewed, most plots of price use a **log scale**, which spreads out the cheaper listings and pulls in the long tail of expensive ones, so differences between groups are easier to see.

Effect of cleaning on price

This plot compares the price distribution before and after cleaning, to show that the cleaning steps removed the unrealistic extreme prices.

```r
stage_data <- rbind(
  data.frame(stage = "Before cleaning", price = priced_all$price),
  data.frame(stage = "After cleaning",  price = airbnb$price)
)
stage_data$stage <- factor(stage_data$stage, levels = c("After cleaning", "Before cleaning"))

ggplot(stage_data, aes(x = price, y = stage)) +
  geom_boxplot(fill = col_main, colour = col_main, alpha = 0.25, width = 0.5, outlier.alpha = 0.3) +
  scale_x_log10(labels = dollar0, breaks = c(30, 100, 300, 1000, 3000, 10000, 30000)) +
  labs(title = "Nightly price before and after cleaning",
       x = "Nightly price (AUD, log scale)", y = NULL)
```

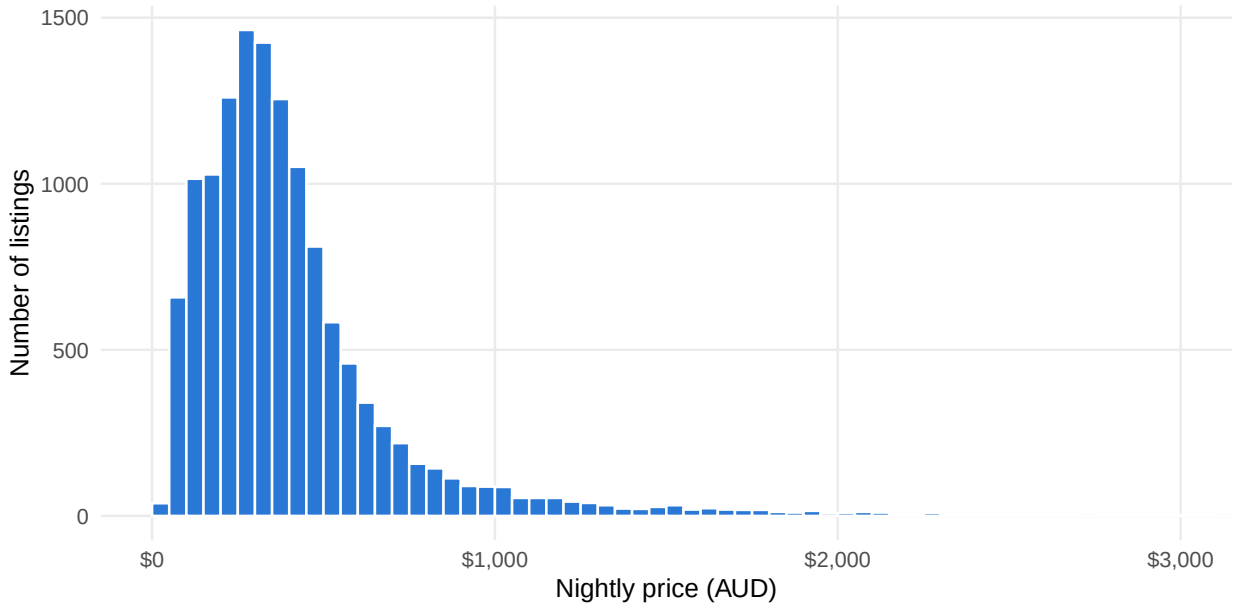


Before cleaning, prices ranged up to $96,888 per night, with many extreme values in both tails. After cleaning, the maximum is $5,077. The box (the middle 50% of listings) is almost unchanged, showing that cleaning removed unrealistic prices without changing the typical listing.

Distribution of price

A histogram of price shows its shape. This matters for choosing which probability distributions to fit in the next section.

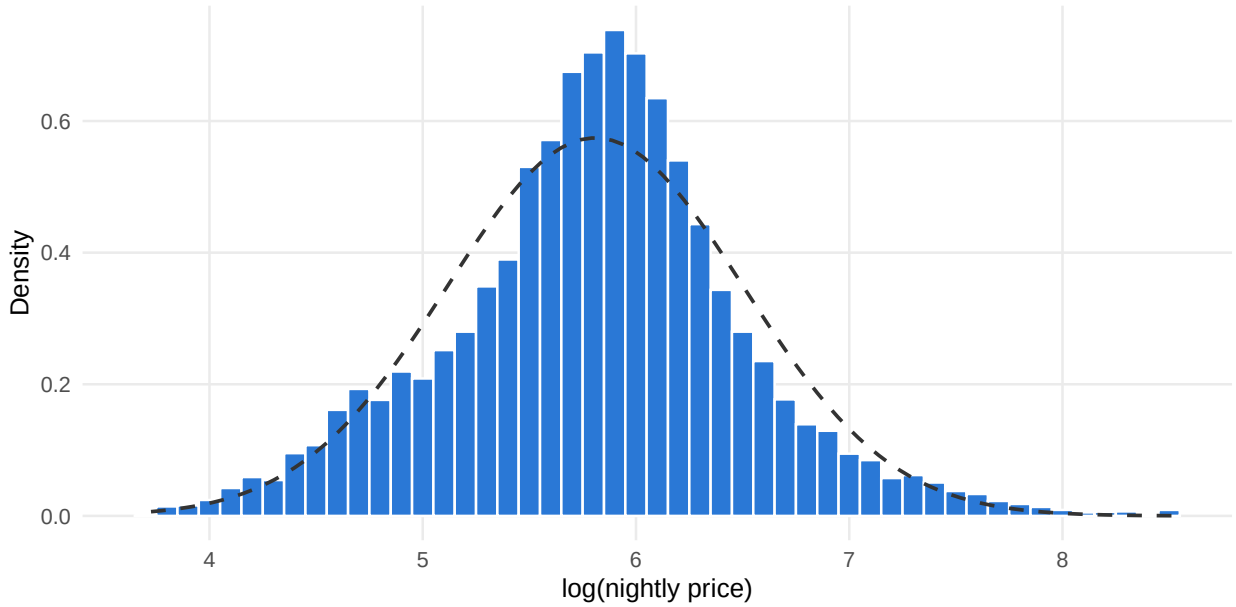
```r
ggplot(airbnb, aes(x = price)) +
  geom_histogram(binwidth = 50, boundary = 0, fill = col_main, colour = "white") +
  scale_x_continuous(labels = dollar0) +
  coord_cartesian(xlim = c(0, 3000)) +
  labs(title = "Distribution of nightly price",
       subtitle = paste0("Skewness = ", round(skewness(airbnb$price), 2),
                         ". Axis cut at $3,000; ", sum(airbnb$price > 3000), " listings lie beyond it.")
       x = "Nightly price (AUD)", y = "Number of listings")
```



Nightly price is strongly right-skewed (skewness = 4.43). Most listings cost between $225 and $495 per night (the middle 50%), but a long tail of expensive listings pulls the mean ($430) well above the median ($341).

The same histogram for log price, with a normal curve added. If log price looks roughly normal, price itself may follow a log-normal distribution.

```
ggplot(airbnb, aes(x = log_price)) +
  geom_histogram(aes(y = after_stat(density)), binwidth = 0.1, fill = col_main, colour = "white") +
  stat_function(fun = dnorm,
                args = list(mean = mean(airbnb$log_price), sd = sd(airbnb$log_price)),
                colour = "grey20", linewidth = 0.8, linetype = "dashed") +
  labs(title = "Distribution of log nightly price",
       subtitle = paste0("Skewness = ", round(skewness(airbnb$log_price), 2),
                         ". Dashed line = normal curve with the same mean and SD."),
       x = "log(nightly price)", y = "Density")
```

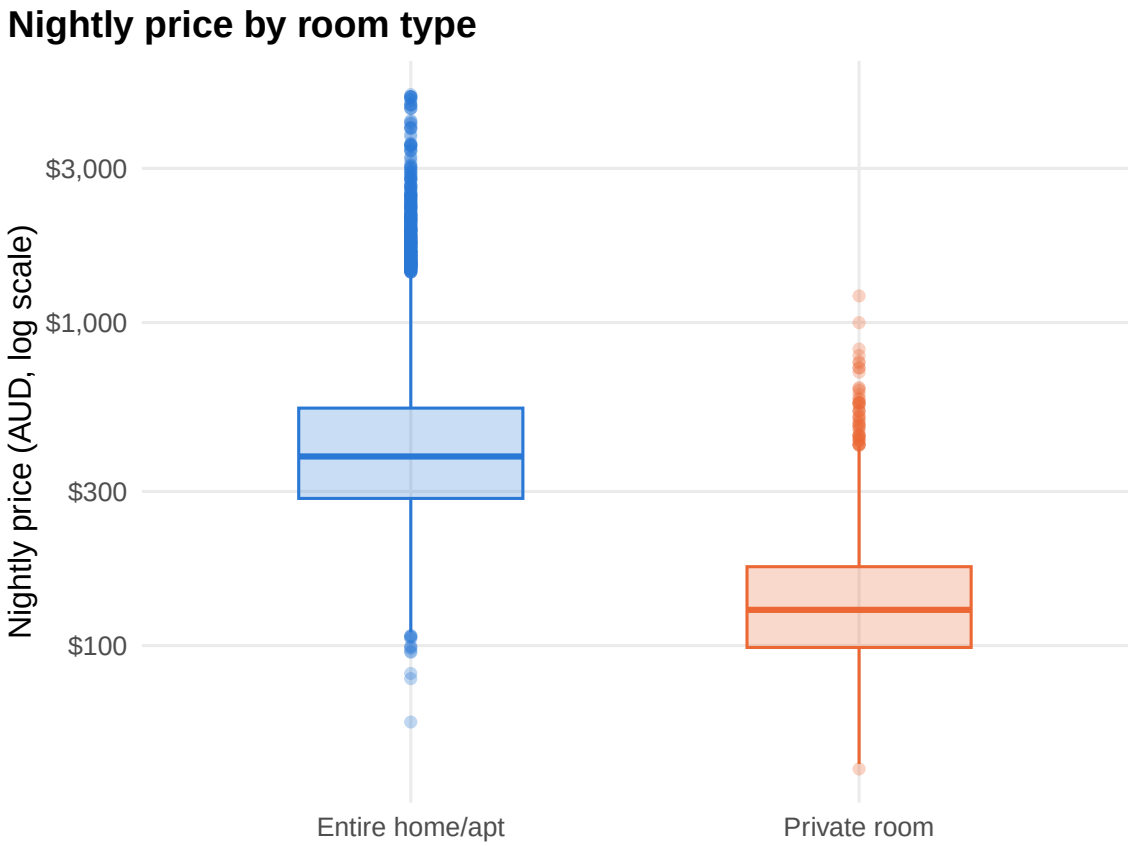


After taking logs, price is close to symmetric (skewness = 0.09) and roughly follows the normal curve. This suggests a log-normal distribution may suit price. The histogram is slightly taller and narrower than the normal curve in the centre, and has a small shoulder on the left from cheaper private rooms.

Price by group

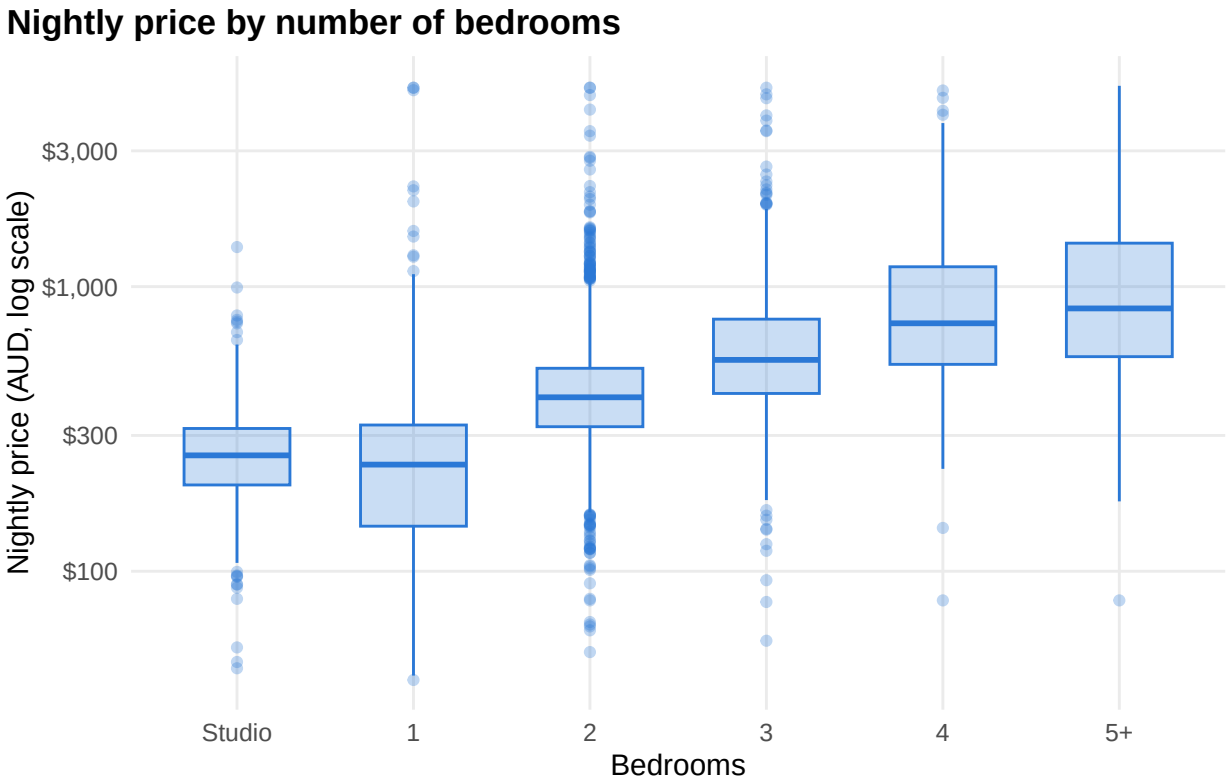
Boxplots compare the median and spread of price across groups. These comparisons suggest research questions for the hypothesis tests.

```
ggplot(airbnb, aes(x = room_type, y = price, fill = room_type, colour = room_type)) +
  geom_boxplot(alpha = 0.25, width = 0.5, outlier.alpha = 0.3) +
  scale_y_log10(labels = dollar0) +
  scale_fill_manual(values = room_cols) +
  scale_colour_manual(values = room_cols) +
  labs(title = "Nightly price by room type",
       x = NULL, y = "Nightly price (AUD, log scale)") +
  theme(legend.position = "none")
```



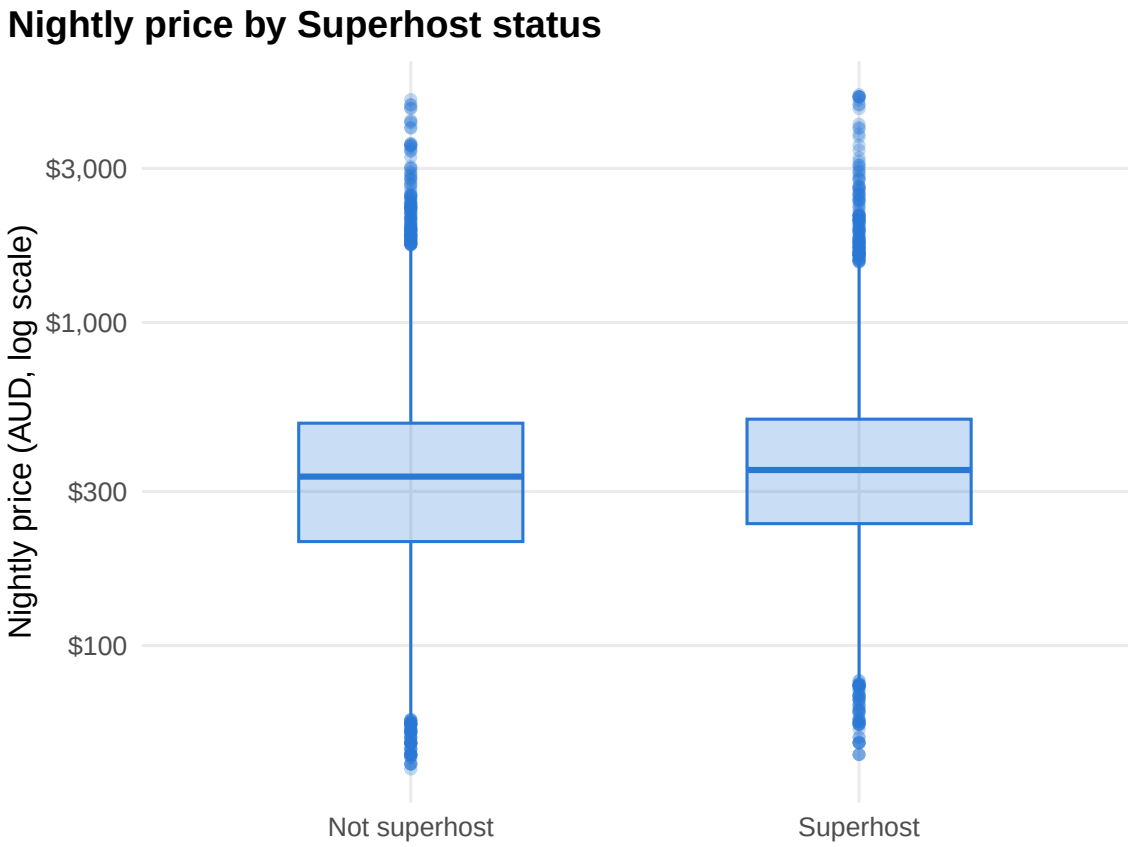
Entire homes are much more expensive than private rooms, with a median of $385 compared with $129. The two boxes barely overlap, making room type the clearest difference in price in the data.

```
ggplot(airbnb, aes(x = bedroom_group, y = price)) +
  geom_boxplot(fill = col_main, colour = col_main, alpha = 0.25, width = 0.6, outlier.alpha = 0.3) +
  scale_y_log10(labels = dollar0) +
  labs(title = "Nightly price by number of bedrooms",
       x = "Bedrooms", y = "Nightly price (AUD, log scale)")
```



Price rises steadily with the number of bedrooms, from a median of $409 for 2 bedrooms to $838 for 5 or more, and the spread also widens for larger properties. Studios (median $255) sit slightly above 1-bedroom listings ($237) because the 1-bedroom group includes private rooms.

```
ggplot(airbnb, aes(x = superhost, y = price)) +
  geom_boxplot(fill = col_main, colour = col_main, alpha = 0.25, width = 0.5, outlier.alpha = 0.3) +
  scale_y_log10(labels = dollar0) +
  labs(title = "Nightly price by Superhost status",
       x = NULL, y = "Nightly price (AUD, log scale)")
```



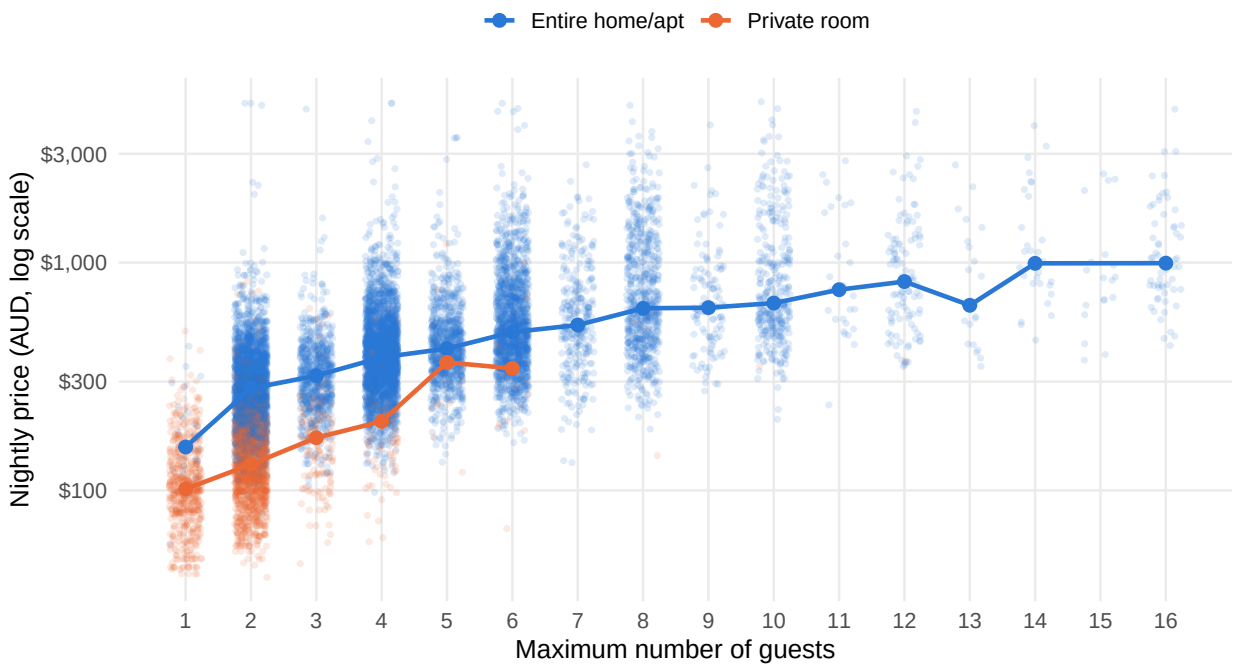
Superhosts and other hosts charge similar prices (medians of $350 and $334), and the boxes overlap almost completely. Whether this small difference is statistically significant is tested in the hypothesis testing section.

Price and number of guests

This shows how price changes with the number of guests a listing can hold, separately for entire homes and private rooms. It helps choose predictors for the regression.

```
median_by_guests <- airbnb %>%
  group_by(room_type, accommodates) %>%
  summarise(median_price = median(price), n = n(), .groups = "drop") %>%
  filter(n >= 20)

ggplot(airbnb, aes(x = accommodates, y = price, colour = room_type)) +
  geom_jitter(width = 0.25, height = 0, alpha = 0.15, size = 0.8) +
  geom_line(data = median_by_guests, aes(y = median_price), linewidth = 1) +
  geom_point(data = median_by_guests, aes(y = median_price), size = 2.5) +
  scale_y_log10(labels = dollar0) +
  scale_x_continuous(breaks = 1:16) +
  scale_colour_manual(values = room_cols, name = NULL) +
  labs(title = "Nightly price by number of guests",
       subtitle = "Dots = listings; line = median price at each guest count (20+ listings)",
       x = "Maximum number of guests", y = "Nightly price (AUD, log scale)") +
  theme(legend.position = "top")
```

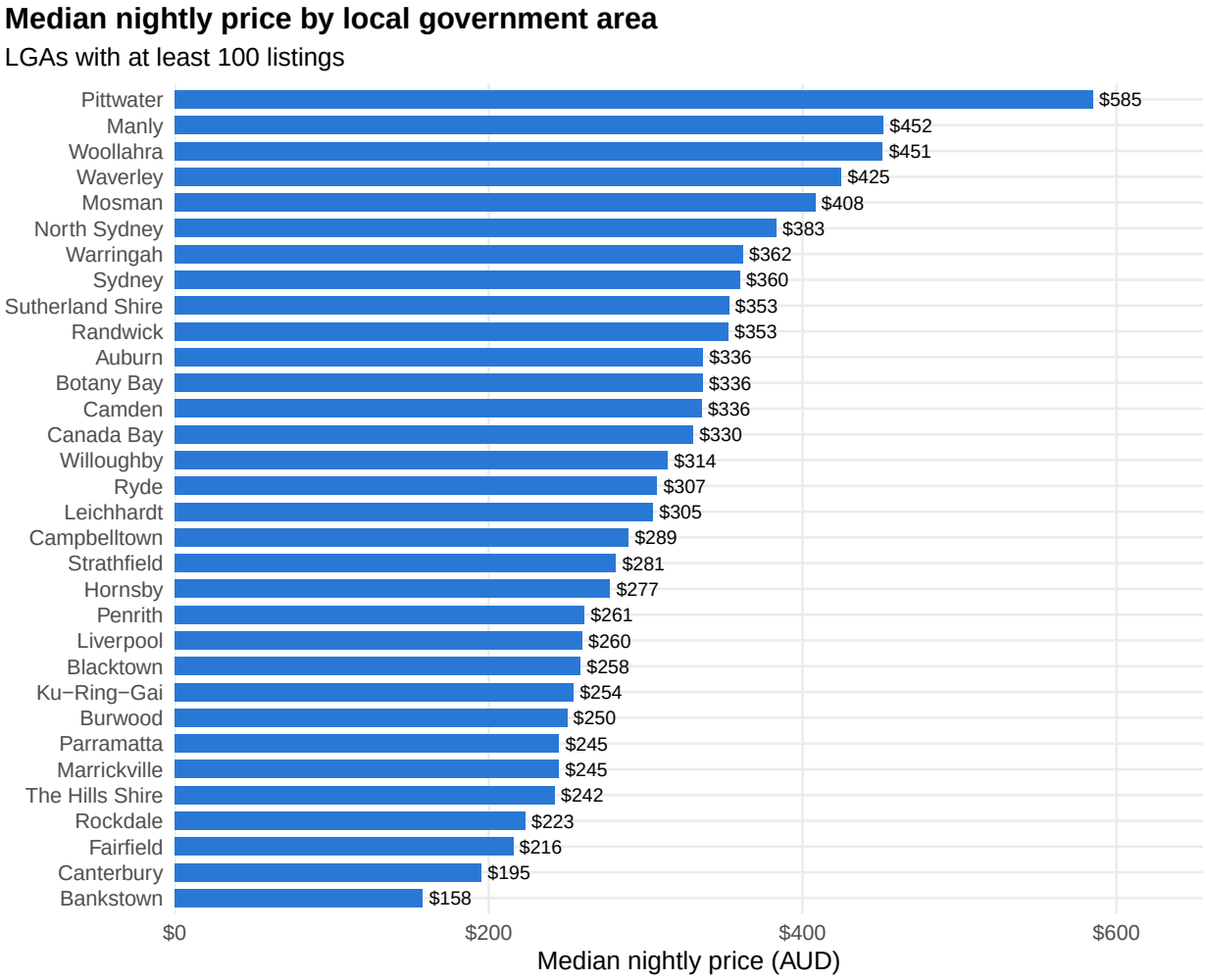


Price increases with the number of guests a listing can hold, for both room types. For entire homes, the median rises from under $200 for 1 guest to around $1,000 for 14–16 guests. At every guest count, entire homes cost more than private rooms, suggesting both variables are useful predictors of price.

Price by location

Median price for each local government area, to show how location affects price.

```
ggplot(price_lga, aes(x = median_price, y = reorder(lga, median_price))) +
  geom_col(fill = col_main, width = 0.7) +
  geom_text(aes(label = dollar0(median_price)), hjust = -0.15, size = 3) +
  scale_x_continuous(labels = dollar0, expand = expansion(mult = c(0, 0.12))) +
  labs(title = "Median nightly price by local government area",
       subtitle = "LGAs with at least 100 listings",
       x = "Median nightly price (AUD)", y = NULL)
```

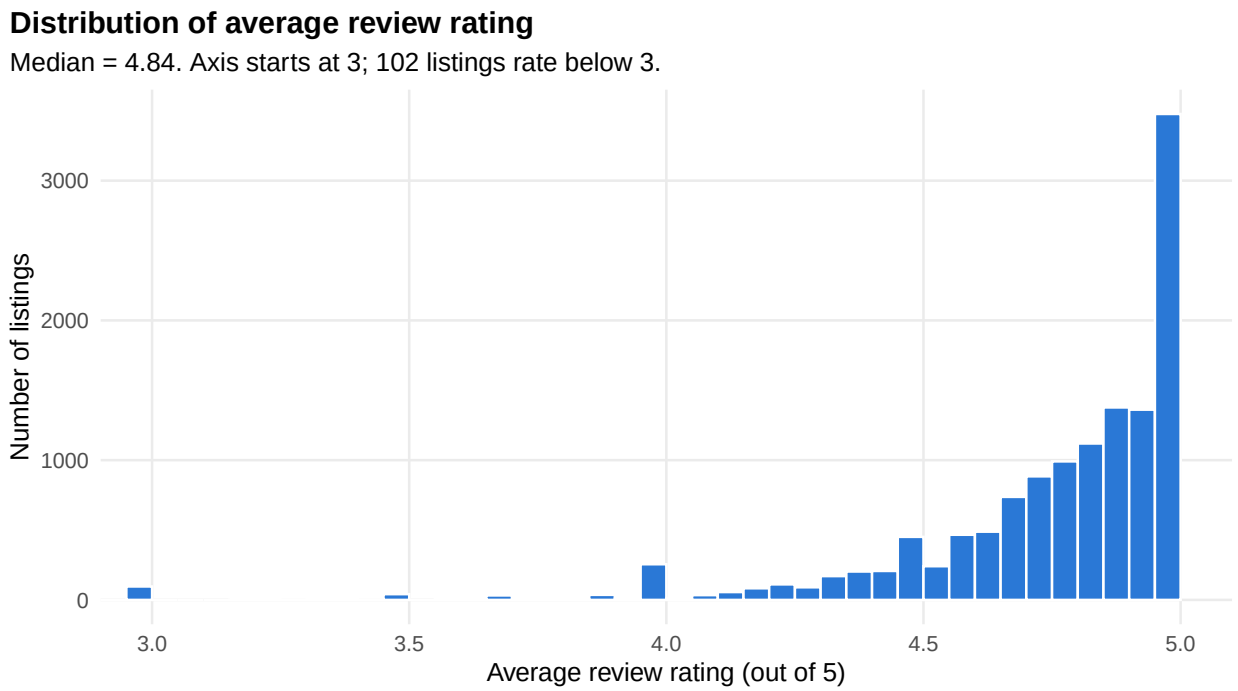


Location has a large effect on price. Coastal and harbourside areas are the most expensive (Pittwater $585, Manly $452, Woollahra $451), while western and south-western areas are the cheapest (Bankstown $158, Canterbury $195). The most expensive area has a median price almost four times that of the cheapest.

Review ratings

The distribution of review ratings, to check its shape before using it in tests.

```
ggplot(airbnb, aes(x = review_rating)) +
  geom_histogram(binwidth = 0.05, boundary = 5, fill = col_main, colour = "white") +
  coord_cartesian(xlim = c(3, 5)) +
  labs(title = "Distribution of average review rating",
       subtitle = paste0("Median = ", median(airbnb$review_rating), ". Axis starts at 3; ",
                         sum(airbnb$review_rating < 3), " listings rate below 3."),
       x = "Average review rating (out of 5)", y = "Number of listings")
```



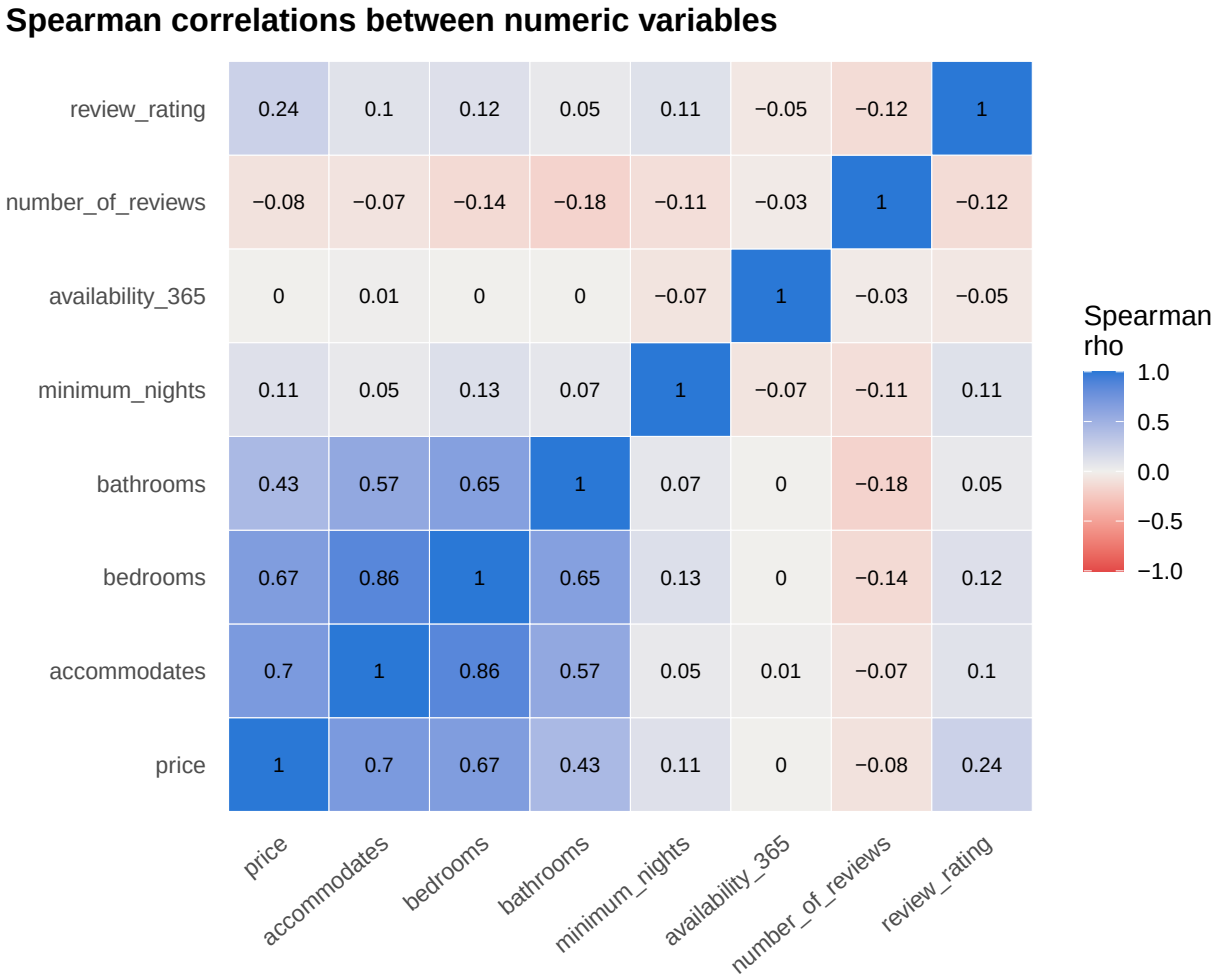
Review ratings are heavily concentrated near 5 (median = 4.84), giving a strong left skew (skewness = -4.51). Only 102 listings rate below 3. The small spikes at exactly 3.0 and 4.0 mostly come from listings with only a few reviews. Because of this shape, nonparametric methods are more suitable than methods assuming normality.

Correlations between variables

A heatmap of the Spearman correlations from Section 5. Blue cells show positive relationships and red cells show negative ones, so the strongest relationships stand out.

```
cor_long <- as.data.frame(as.table(cor_mat))
names(cor_long) <- c("var1", "var2", "rho")

ggplot(cor_long, aes(x = var1, y = var2, fill = rho)) +
  geom_tile(colour = "white") +
  geom_text(aes(label = round(rho, 2)), size = 3) +
  scale_fill_gradient2(low = "#e34948", mid = "#f0efec", high = "#2a78d6",
                       midpoint = 0, limits = c(-1, 1), name = "Spearman\nrho") +
  labs(title = "Spearman correlations between numeric variables", x = NULL, y = NULL) +
  theme(axis.text.x = element_text(angle = 40, hjust = 1), panel.grid = element_blank())
```



Price is most strongly related to the number of guests (rho = 0.70) and bedrooms (rho = 0.67), and moderately to bathrooms (rho = 0.43). It has almost no relationship with availability or number of reviews. Accommodates and bedrooms are also strongly correlated with each other (rho = 0.86), so including both in a regression model may cause multicollinearity.

7. Distribution fitting

Nightly price takes only positive values and is strongly right-skewed (Section 5). We therefore fit two distributions that are defined for positive values and allow right skew: the **Gamma** distribution and the **log-normal** distribution. For each, we estimate the parameters by the method of moments (MoM) and maximum likelihood (MLE), derive the Cramér-Rao lower bound, build confidence intervals, and compare the two models using AIC and BIC.

7.1 The two models

Gamma distribution with shape $\alpha > 0$ and rate $\beta > 0$:

$$f(x;\alpha,\beta) = \frac{\beta^{\alpha}}{\Gamma(\alpha)}\, x^{\alpha-1} e^{-\beta x}, \qquad x > 0$$

$$E(X) = \frac{\alpha}{\beta}, \qquad \mathrm{Var}(X) = \frac{\alpha}{\beta^2}$$

Log-normal distribution with parameters μ and $\sigma > 0$, where $\log X \sim N(\mu, \sigma^2)$:

$$f(x; \mu, \sigma) = \frac{1}{x\sigma\sqrt{2\pi}} \exp\left\{-\frac{(\log x - \mu)^2}{2\sigma^2}\right\}, \qquad x > 0$$

$$E(X) = e^{\mu + \sigma^2/2}, \qquad \mathrm{Var}(X) = \left(e^{\sigma^2} - 1\right) e^{2\mu + \sigma^2}$$

7.2 Method of moments

The method of moments sets the population mean and variance equal to the sample mean $\bar{x}$ and sample variance $s^2 = \frac{1}{n}\sum_{i=1}^{n}(x_i - \bar{x})^2$, and solves for the parameters. This is equivalent to equating the first two moments $E[X]$ and $E[X^2]$ to the sample moments $\frac{1}{n}\sum x_i$ and $\frac{1}{n}\sum x_i^2$, since $s^2 = \frac{1}{n}\sum x_i^2 - \bar{x}^2$.

Gamma: from $\alpha/\beta = \bar{x}$ and $\alpha/\beta^2 = s^2$, dividing the first by the second gives $\beta = \bar{x}/s^2$, and substituting back gives

$$\hat{\alpha}_{\mathrm{MoM}} = \frac{\bar{x}^2}{s^2}, \qquad \hat{\beta}_{\mathrm{MoM}} = \frac{\bar{x}}{s^2}$$

Log-normal: dividing the variance by the squared mean gives $\mathrm{Var}(X)/E(X)^2 = e^{\sigma^2} - 1 = s^2/\bar{x}^2$, so

$$\hat{\sigma}^2_{\mathrm{MoM}} = \log\left(1 + \frac{s^2}{\bar{x}^2}\right), \qquad \hat{\mu}_{\mathrm{MoM}} = \log\bar{x} - \frac{\hat{\sigma}^2_{\mathrm{MoM}}}{2}$$

```
x    <- airbnb$price
n    <- length(x)
xbar <- mean(x)
s2   <- mean((x - xbar)^2)    # sample variance with divisor n
c(n = n, mean = xbar, variance = s2)
```

```
##           n        mean    variance
##  13219.0000    429.5498 154711.3578
```

```
# Gamma
alpha_mom <- xbar^2 / s2
beta_mom  <- xbar / s2

# Log-normal
sigma2_mom <- log(1 + s2 / xbar^2)
mu_mom     <- log(xbar) - sigma2_mom / 2
sigma_mom  <- sqrt(sigma2_mom)

c(alpha = alpha_mom, beta = beta_mom, mu = mu_mom, sigma = sigma_mom)
```

```
##       alpha        beta          mu       sigma
## 1.192627484 0.002776459 5.758266774 0.780347196
```

7.3 Maximum likelihood

Gamma. The log-likelihood for a sample $x_1, \ldots, x_n$ is

$$\ell(\alpha, \beta) = n\alpha \log \beta - n \log \Gamma(\alpha) + (\alpha - 1) \sum_{i=1}^{n} \log x_i - \beta \sum_{i=1}^{n} x_i$$

Setting the partial derivatives (score equations) to zero:

$$\frac{\partial \ell}{\partial \beta} = \frac{n\alpha}{\beta} - \sum_{i=1}^{n} x_i = 0 \quad \Rightarrow \quad \hat{\beta} = \frac{\hat{\alpha}}{\bar{x}}$$

$$\frac{\partial \ell}{\partial \alpha} = n \log \beta - n\psi(\alpha) + \sum_{i=1}^{n} \log x_i = 0$$

where $\psi(\alpha) = \frac{d}{d\alpha} \log \Gamma(\alpha)$ is the digamma function. Substituting $\hat{\beta} = \hat{\alpha}/\bar{x}$ gives

$$\log \hat{\alpha} - \psi(\hat{\alpha}) = \log \bar{x} - \frac{1}{n} \sum_{i=1}^{n} \log x_i$$

This equation has no closed-form solution, so the log-likelihood is maximised numerically with `optim()`, using the bounded version of the BFGS algorithm. As a check, the equation above is also solved directly with `uniroot()`.

```
# Log-likelihood of the Gamma model
loglik_gamma <- function(par) {
  sum(dgamma(x, shape = par[1], rate = par[2], log = TRUE))
}

fit_gamma <- optim(par = c(alpha_mom, beta_mom),        # start at the MoM estimates
                   fn = loglik_gamma,
                   method = "L-BFGS-B",                 # BFGS with bounds
                   lower = c(1e-6, 1e-8),               # keep both parameters positive
                   control = list(fnscale = -1,         # -1 means maximise
                                  parscale = c(alpha_mom, beta_mom)))

fit_gamma$convergence    # 0 means it converged
```

```
## [1] 0
```

```
alpha_mle <- fit_gamma$par[1]
beta_mle  <- fit_gamma$par[2]
c(alpha = alpha_mle, beta = beta_mle)
```

```
##       alpha        beta
## 2.122897161 0.004942145
```

```
# Check: solve the score equation for alpha directly
score_alpha <- function(a) log(a) - digamma(a) - (log(xbar) - mean(log(x)))
uniroot(score_alpha, interval = c(0.01, 100))$root
```

```
## [1] 2.122896
```

Log-normal. The log-likelihood is

$$\ell(\mu, \sigma^2) = -\sum_{i=1}^{n} \log x_i - \frac{n}{2}\log(2\pi\sigma^2) - \frac{1}{2\sigma^2}\sum_{i=1}^{n}(\log x_i - \mu)^2$$

Setting the partial derivatives to zero:

$$\frac{\partial \ell}{\partial \mu} = \frac{1}{\sigma^2}\sum_{i=1}^{n}(\log x_i - \mu) = 0 \quad \Rightarrow \quad \hat{\mu} = \frac{1}{n}\sum_{i=1}^{n}\log x_i$$

$$\frac{\partial \ell}{\partial \sigma^2} = -\frac{n}{2\sigma^2} + \frac{1}{2\sigma^4}\sum_{i=1}^{n}(\log x_i - \mu)^2 = 0 \quad \Rightarrow \quad \hat{\sigma}^2 = \frac{1}{n}\sum_{i=1}^{n}(\log x_i - \hat{\mu})^2$$

These have closed-form solutions: the mean and variance (divisor n) of log price, so no numerical method is needed. By the invariance property of MLEs, $\hat{\sigma} = \sqrt{\hat{\sigma}^2}$.

```
mu_mle     <- mean(log(x))
sigma2_mle <- mean((log(x) - mu_mle)^2)
sigma_mle  <- sqrt(sigma2_mle)
c(mu = mu_mle, sigma = sigma_mle)
```

```
##       mu    sigma
## 5.809094 0.694593
```

7.4 Summary of estimates

```
estimates <- data.frame(
  distribution = c("Gamma", "Gamma", "Log-normal", "Log-normal"),
  parameter    = c("alpha (shape)", "beta (rate)", "mu", "sigma"),
  MoM          = c(alpha_mom, beta_mom, mu_mom, sigma_mom),
  MLE          = c(alpha_mle, beta_mle, mu_mle, sigma_mle)
)
knitr::kable(estimates, digits = 6)
```

distribution	parameter	MoM	MLE
Gamma	alpha (shape)	1.192627	2.122897
Gamma	beta (rate)	0.002776	0.004942
Log-normal	mu	5.758267	5.809094
Log-normal	sigma	0.780347	0.694593

For the Gamma distribution, the MoM and MLE estimates differ substantially ($\hat{\alpha} = 1.19$ by MoM vs 2.12 by MLE). The MoM estimates rely on the sample variance, which is heavily inflated by the long right tail of expensive listings. The MLE uses the full likelihood of the data. For the log-normal distribution, the two methods give closer results. The MLEs are preferred because they are consistent, asymptotically efficient and asymptotically normal.

7.5 Fisher information

The Fisher information is the negative expected Hessian (matrix of second derivatives) of the log-likelihood, $I(\theta) = -E\left[\frac{\partial^2 \ell}{\partial\theta\,\partial\theta^T}\right]$.

Gamma. Differentiating the score equations again:

$$\frac{\partial^2 \ell}{\partial \alpha^2} = -n\psi'(\alpha), \qquad \frac{\partial^2 \ell}{\partial \alpha\,\partial \beta} = \frac{n}{\beta}, \qquad \frac{\partial^2 \ell}{\partial \beta^2} = -\frac{n\alpha}{\beta^2}$$

where $\psi'(\alpha)$ is the trigamma function. None of these depend on the data, so

$$I(\alpha, \beta) = n \begin{pmatrix} \psi'(\alpha) & -1/\beta \\ -1/\beta & \alpha/\beta^2 \end{pmatrix}$$

Log-normal. Writing the log-likelihood in terms of σ, $\ell(\mu, \sigma) = -\sum \log x_i - n \log \sigma - \frac{n}{2}\log(2\pi) - \frac{1}{2\sigma^2}\sum(\log x_i - \mu)^2$, the second derivatives are

$$\frac{\partial^2 \ell}{\partial \mu^2} = -\frac{n}{\sigma^2}, \qquad \frac{\partial^2 \ell}{\partial \mu\,\partial \sigma} = -\frac{2}{\sigma^3}\sum_{i=1}^{n}(\log x_i - \mu), \qquad \frac{\partial^2 \ell}{\partial \sigma^2} = \frac{n}{\sigma^2} - \frac{3}{\sigma^4}\sum_{i=1}^{n}(\log x_i - \mu)^2$$

Since $E[\log X_i - \mu] = 0$ and $E[(\log X_i - \mu)^2] = \sigma^2$, taking expectations gives

$$I(\mu, \sigma) = \begin{pmatrix} n/\sigma^2 & 0 \\ 0 & 2n/\sigma^2 \end{pmatrix}$$

```
# Gamma: Fisher information evaluated at the MLEs
I_gamma <- n * matrix(c(trigamma(alpha_mle), -1 / beta_mle,
                        -1 / beta_mle,       alpha_mle / beta_mle^2),
                      nrow = 2)

# Log-normal: Fisher information evaluated at the MLEs
I_lnorm <- matrix(c(n / sigma_mle^2, 0,
                    0,               2 * n / sigma_mle^2),
                  nrow = 2)

I_gamma
```

```
##              [,1]       [,2]
## [1,]     7914.798   -2674749
## [2,] -2674749.371 1148937863
```

```
I_lnorm
```

```
##          [,1]     [,2]
## [1,] 27399.19     0.00
## [2,]     0.00 54798.38
```

The off-diagonal terms of the Gamma information matrix are non-zero, so the estimates of α and β are correlated. For the log-normal model the matrix is diagonal, so μ and σ are estimated independently of each other.

7.6 Cramér-Rao lower bound

For an unbiased estimator $\hat{\theta}$ of a single parameter θ, the Cramér-Rao lower bound (CRLB) states that $\text{Var}(\hat{\theta}) \geq 1/I(\theta)$, where $I(\theta)$ is the Fisher information for θ alone, treating the other parameter as known.

Gamma, rate β (with α known). From above, $I(\beta) = -E\left[\frac{\partial^2 \ell}{\partial \beta^2}\right] = \frac{n\alpha}{\beta^2}$, so

$$\text{Var}(\hat{\beta}) \geq \frac{\beta^2}{n\alpha}$$

Log-normal, μ (with σ known). $I(\mu) = -E\left[\frac{\partial^2 \ell}{\partial \mu^2}\right] = \frac{n}{\sigma^2}$, so

$$\text{Var}(\hat{\mu}) \geq \frac{\sigma^2}{n}$$

The MLE $\hat{\mu} = \frac{1}{n}\sum \log x_i$ is unbiased, since $E[\log X_i] = \mu$, and has variance $\text{Var}(\hat{\mu}) = \sigma^2/n$. Its variance equals the CRLB, so $\hat{\mu}$ is the minimum variance unbiased estimator (MVUE) of μ.

```
crlb_beta <- beta_mle^2 / (n * alpha_mle)
crlb_mu   <- sigma_mle^2 / n

# For comparison: asymptotic variance of each MLE when BOTH parameters are unknown
var_beta_mle <- solve(I_gamma)[2, 2]
var_mu_mle   <- solve(I_lnorm)[1, 1]

crlb <- data.frame(
  parameter    = c("beta (Gamma)", "mu (Log-normal)"),
  CRLB         = format(c(crlb_beta, crlb_mu), digits = 4),
  MLE_variance = format(c(var_beta_mle, var_mu_mle), digits = 4),
  ratio        = round(c(var_beta_mle / crlb_beta, var_mu_mle / crlb_mu), 3)
)
knitr::kable(crlb)
```

parameter	CRLB	MLE_variance	ratio
beta (Gamma)	8.704e-10	4.081e-09	4.689
mu (Log-normal)	3.650e-05	3.650e-05	1.000

For μ, the asymptotic variance of the MLE equals the CRLB (ratio = 1): even when σ is unknown, $\hat{\mu}$ is as precise as any unbiased estimator can be, because the information matrix is diagonal. For β, the asymptotic variance of the MLE when α must also be estimated is about 4.7 times the CRLB with α known. This is because α and β are strongly correlated, so uncertainty about α adds to the uncertainty about β.

7.7 Confidence intervals

As $n \to \infty$, the MLE is approximately normal, $\hat{\theta} \sim N\left(\theta, I(\theta)^{-1}\right)$. The asymptotic standard errors are the square roots of the diagonal of $I(\hat{\theta})^{-1}$, and an approximate 95% confidence interval for each parameter is

$$\hat{\theta} \pm z_{0.975} \times \text{SE}(\hat{\theta}), \qquad z_{0.975} = 1.96$$

```r
se_gamma <- sqrt(diag(solve(I_gamma)))
se_lnorm <- sqrt(diag(solve(I_lnorm)))

z <- qnorm(0.975)

ci <- data.frame(
  distribution = c("Gamma", "Gamma", "Log-normal", "Log-normal"),
  parameter    = c("alpha", "beta", "mu", "sigma"),
  estimate     = c(alpha_mle, beta_mle, mu_mle, sigma_mle),
  SE           = c(se_gamma, se_lnorm)
)
ci$lower <- ci$estimate - z * ci$SE
ci$upper <- ci$estimate + z * ci$SE
knitr::kable(ci, digits = 6)
```

distribution	parameter	estimate	SE	lower	upper
Gamma	alpha	2.122897	0.024340	2.075191	2.170603
Gamma	beta	0.004942	0.000064	0.004817	0.005067
Log-normal	mu	5.809094	0.006041	5.797253	5.820934
Log-normal	sigma	0.694593	0.004272	0.686220	0.702966

```r
# The median of a log-normal distribution is exp(mu), so the CI for mu
# gives a CI for the median nightly price
exp(c(median = mu_mle, lower = mu_mle - z * se_lnorm[1], upper = mu_mle + z * se_lnorm[1]))
```

```
##   median    lower    upper
## 333.3169 329.3934 337.2871
```

All intervals are very narrow because the sample is large (n = 13,219), so the parameters are estimated precisely. Since the median of a log-normal distribution is e^{μ}, the interval for μ implies a median nightly price between \$329 and \$337. This is slightly below the observed sample median of \$341, an early sign that the log-normal model does not fit perfectly.

7.8 Model comparison: AIC and BIC

The Akaike Information Criterion (AIC; Akaike, 1974) and the Bayesian Information Criterion (BIC; Schwarz, 1978) compare models by balancing goodness of fit against model complexity:

$$\text{AIC} = -2\ell(\hat{\theta}) + 2k, \qquad \text{BIC} = -2\ell(\hat{\theta}) + k \log n$$

where $\ell(\hat{\theta})$ is the maximised log-likelihood, k is the number of parameters and n is the sample size. Lower values indicate a better model. BIC penalises extra parameters more heavily than AIC when n is large. As a rule of thumb, a difference in AIC of more than 10 is strong evidence for the model with the lower value (Burnham & Anderson, 2002).

```r
ll_gamma <- fit_gamma$value
ll_lnorm <- sum(dlnorm(x, meanlog = mu_mle, sdlog = sigma_mle, log = TRUE))
k <- 2
```

```r
comparison <- data.frame(
  model  = c("Gamma", "Log-normal"),
  loglik = c(ll_gamma, ll_lnorm),
  AIC    = -2 * c(ll_gamma, ll_lnorm) + 2 * k,
  BIC    = -2 * c(ll_gamma, ll_lnorm) + k * log(n)
)
knitr::kable(comparison, digits = 2)
```

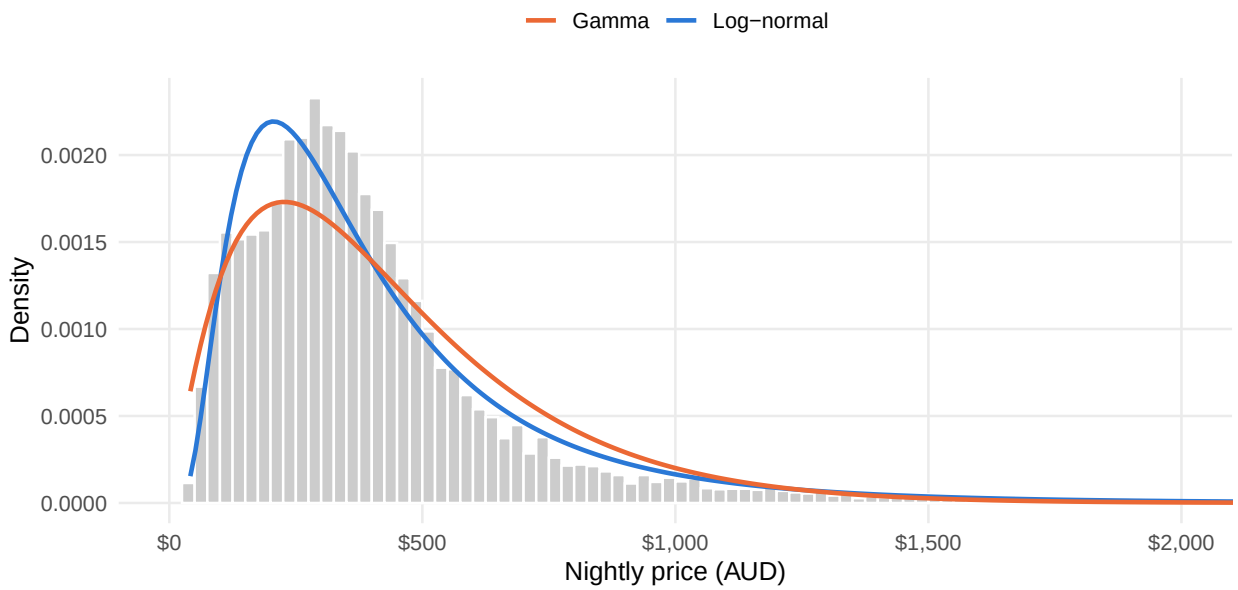
model	loglik	AIC	BIC
Gamma	-91595.53	183195.1	183210.0
Log-normal	-90729.97	181463.9	181478.9

The log-normal model has a higher log-likelihood and lower AIC and BIC than the Gamma model, with a difference of about 1,731 on both criteria. Because both models have two parameters, the penalty terms are equal and the comparison depends only on the log-likelihood. A difference this large is overwhelming evidence in favour of the log-normal model.

7.9 Visual check of the fitted models

The fitted densities are drawn over the histogram of price to compare their shapes with the data.

```r
ggplot(airbnb, aes(x = price)) +
  geom_histogram(aes(y = after_stat(density)), binwidth = 25, boundary = 0,
                 fill = "grey80", colour = "white") +
  stat_function(aes(colour = "Log-normal"), fun = dlnorm,
                args = list(meanlog = mu_mle, sdlog = sigma_mle), n = 500, linewidth = 1) +
  stat_function(aes(colour = "Gamma"), fun = dgamma,
                args = list(shape = alpha_mle, rate = beta_mle), n = 500, linewidth = 1) +
  scale_colour_manual(values = c("Log-normal" = "#2a78d6", "Gamma" = "#eb6834"), name = NULL) +
  scale_x_continuous(labels = dollar0) +
  coord_cartesian(xlim = c(0, 2000)) +
  labs(title = "Fitted Gamma and log-normal densities",
       subtitle = "Maximum likelihood fits. Axis cut at $2,000.",
       x = "Nightly price (AUD)", y = "Density") +
  theme(legend.position = "top")
```



Both models capture the right skew, but neither matches the data perfectly. The Gamma curve is too flat: it puts too much probability on very cheap listings and too little on typical prices between about $250 and $450. The log-normal curve follows the data more closely, although its peak sits at a lower price (around $200) than the peak of the histogram (around $300).

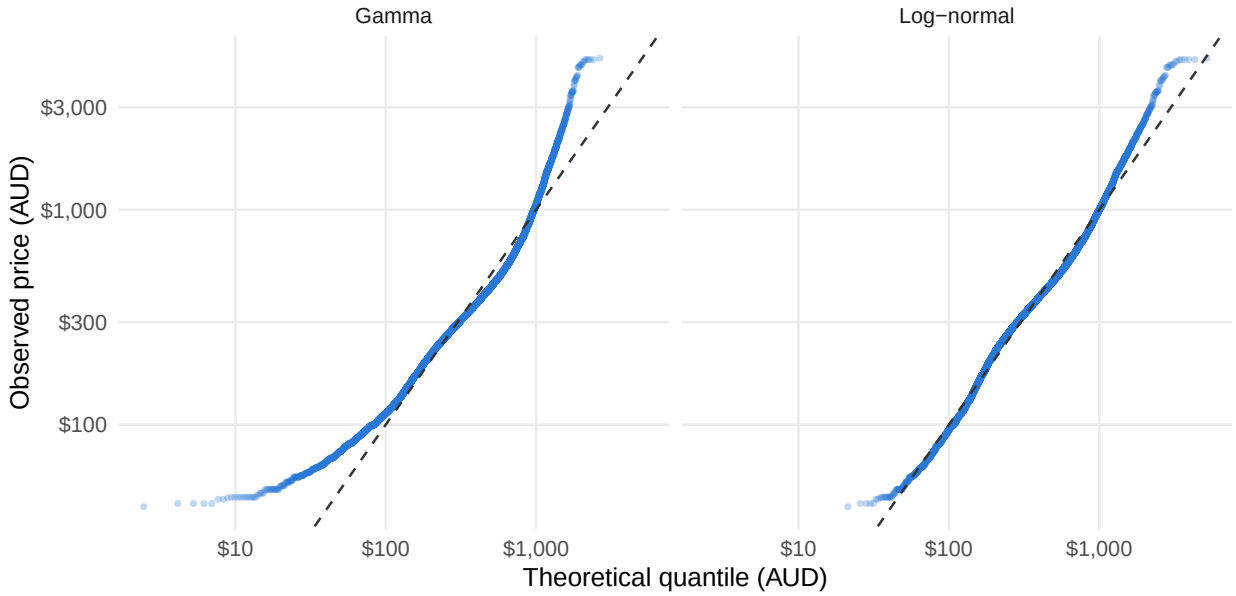
```
p <- ppoints(n)     # probabilities (i - 0.5) / n

qq <- rbind(
  data.frame(model = "Gamma",      theoretical = qgamma(p, shape = alpha_mle, rate = beta_mle), sample =
  data.frame(model = "Log-normal", theoretical = qlnorm(p, meanlog = mu_mle, sdlog = sigma_mle), sample
)

ggplot(qq, aes(x = theoretical, y = sample)) +
  geom_point(colour = col_main, alpha = 0.3, size = 0.8) +
  geom_abline(slope = 1, intercept = 0, colour = "grey20", linetype = "dashed") +
  scale_x_log10(labels = dollar0) +
  scale_y_log10(labels = dollar0) +
  facet_wrap(~ model) +
  labs(title = "Q-Q plots of nightly price against the fitted models",
       subtitle = "Points on the dashed line mean the model matches the data. Log scales.",
       x = "Theoretical quantile (AUD)", y = "Observed price (AUD)")
```

Q−Q plots of nightly price against the fitted models

Points on the dashed line mean the model matches the data. Log scales.



For the log-normal model, the points lie close to the dashed line across almost the entire range, with only small departures for the most expensive listings (above about $2,000). For the Gamma model, the points bend away from the line at both ends: the model predicts some listings cheaper than $20, which do not exist in the data, and greatly underestimates prices above $1,000. The log-normal model therefore describes the long right tail of prices much better.

7.10 Model selection

Based on AIC and BIC, the **log-normal distribution** is selected as the better model for nightly price. It has a much lower AIC and BIC (a difference of about 1,731), and the Q-Q plot shows that it fits the data closely across almost the whole range, while the Gamma model underestimates the long right tail. This agrees with the EDA, where log price was close to symmetric (skewness = 0.09).

The log-normal model is not perfect. The data are more concentrated around the centre than the model predicts, which is likely because the sample combines two different markets (entire homes and private rooms). Fitting a separate model for each room type could improve the fit.